

What's New in Java 9?

After a couple of delays, Java 9 is finally here. Three years after Java 8, it comes with some of the biggest changes to the Java platform, adding more than 150 new features. In this article, we will cover some of the major changes.

The module system in Java 9 is one of the most talked about and awaited feature. So, what's it all about?

Java provides an encapsulation mechanism with the help of access modifiers, and any adequately sized Java project will have classes across multiple packages. If the project is large enough, you will have dependencies spanning multiple JAR files. Two problems with this approach are:

- Not all the classes declared 'public' are part of the APIs that you might want to expose.
- What if two JARs have the same class declared under the same package (split packages)? This can lead to class loading issues that you will get to know about only at the runtime.

To solve these issues, Java 9 has introduced the concept of modules. A module is a higher-level abstraction over packages. To create a module, you need to add a *module-info.java* file to your project, and explicitly define the dependencies on other modules using the *requires* keyword as well as the packages from which you want to expose the public classes using the *exports* keyword. Do note that other public classes inside any package(s) will not be accessible outside this module, other than the ones inside the exported packages. A sample package definition will look like what follows:

```
module my-module {
    exports com.example.java9.classes_to_be_exposed_as_apis;
    requires java.base;
}
```

JLink: For creating smaller Java runtimes

Java runtime has evolved considerably since its inception, and thousands of classes have been added to it to improve functionality, which has turned it into a huge monolith. Not all the features are used by most of the programs at any given time. For example, until Java 8, you could create a small 'Hello world' program that only used classes from the *java.lang* package, but the runtime would still include all the features (xml parsers, logging APIs, SQL functionality, etc) available in Java.

The JDK itself was rewritten using the module system, so there is no single *rt.jar* file that has all the features of Java; and it has been refactored into small individual modules. This means that you can create custom runtimes that only include the parts of JDK that are used in your module. Once you have

a Java module JAR file, you can use the JLink tool in the JDK to generate the runtime:

```
jlink --module-path <modulepath> --add-modules
<modules> --limit-modules <modules> --output <path>
```

You can find the dependent modules required by your module using the *jdeps* tool. Just to give you some perspective, the standard JDK 9 runtime is around 430MB and for a simple 'Hello World' application, the runtime generated using the above command is around 24MB, which is great for devices with memory restrictions, for IoT applications and for custom runtimes to run inside containers like Docker.

Private methods inside interfaces

Java 8 introduced features with which you could have default function implementations inside an interface but all the default implementations behaved like public methods. Well, now you can have private methods inside the interfaces that can be used by the default methods for better code organisation.

```
public interface Car {
    void setCarColor(String color); // normal interface method
    default void startCar () {
        // default method
        System.out.println("Car is starting...");
        startEngine();
        System.out.println("Car started! ");
    }

    private void startEngine() {
        // Private method not available outside the interface as
        // an API
        System.out.println("Engine Started ");
    }
}
```

JShell: The interactive Java REPL

Like Python, PHP and several other languages, now Java too features an interactive Read-Eval-Print-Loop. To get started, just type 'jshell' in the console and start typing your Java code:

```
C:\Program Files\Java\jdk-9\bin>jshell
```

